

CondoHub

Guia de Desenvolvimento do Banco de Dados PostgreSQL no Supabase

Versão: 1.0 | Data: 24/02/2026

Este documento descreve como criar, migrar, proteger e validar o banco do CondoHub no Supabase (Postgres), baseado na modelagem oficial do projeto.

1. Premissas e decisões

Stack: PostgreSQL (Supabase) + backend (API) como única fonte de verdade.

Multi-instância (tenant): toda entidade de negócio carrega `instance_id` e as queries do backend sempre filtram por esse campo (resolvido a partir de `instance_key` na rota).

Segurança: o front não acessa o Postgres direto. Recomenda-se bloquear acesso via chaves `anon/authenticated` e operar via `service_role` ou conexão direta do backend.

Uploads: arquivos ficam no Supabase Storage (bucket attachments), e o banco guarda metadados em `attachments` e vínculos em `attachment_links`.

2. Setup do projeto no Supabase

2.1 Criar projeto

No painel do Supabase: crie um projeto, defina senha forte do Postgres e escolha região próxima do seu público.

Guarde com cuidado:

- **Project ref** (ex.: `nvtxysxvriperzluynky`)
- **Database password**
- **SUPABASE_URL** e **SERVICE_ROLE_KEY**

2.2 Extensões necessárias

O schema usa UUIDs e e-mails case-insensitive. Rode no SQL Editor:

```
CREATE EXTENSION IF NOT EXISTS pgcrypto; -- gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS citext;   -- e-mails case-insensitive
```

2.3 Storage (bucket attachments)

Crie um bucket chamado `attachments` e marque como **Private**.

Recomendação de path (padrão do backend): `{instance_id}/{yyyy}/{mm}/{uuid}.{ext}`.

O backend deve gerar URLs assinadas para download e validar que o path pertence ao tenant.

3. Estrutura do schema

O schema é dividido em blocos:

Bloco	Tabelas (principais)	Observações
Admin Global (SaaS)	plans, instances	Planos e instâncias; instance_key é UNIQUE
IAM (Auth)	users, user_credentials, sessions	JWT curto + refresh rotativo; token_version em users
Fundacionais	condo_profiles, blocks, units, invites	Estrutura do condomínio e convites
Uploads	attachments, attachment_links	Metadados + link para entidades
Auditoria	audit_logs	Append-only; update/delete bloqueados por trigger
Push	push_tokens	Tokens de push por usuário/instância
Comunicação	announcements, announcement_acks, channels, channel_posts, channels_messages, announcements_attachments, announcements_attachments_attachments	Notificações, mensagens, comentários, etc.
Tickets	tickets, ticket_messages, ticket_status_history	Chamados com SLA (due_at) e histórico
Entregas	deliveries, delivery_events, delivery_turns	Fila, eventos e turnos; 1 turno ativo por user/instância

Notas importantes de integridade

- **users.email**: unicidade global para ADMIN_GLOBAL (instance_id NULL) e unicidade por instância para os demais.
- **invites**: check constraint garante que convite de MORADOR exige unit_id, e outros papéis não podem ter unit_id.
- **delivery_turns**: unique parcial garante 1 turno ativo (ended_at IS NULL) por (instance_id, user_id).
- **Soft delete**: blocks/units/channels/announcements usam archived_at; posts/comments usam deleted_at.

4. Migrations e como executar

Foram gerados 3 arquivos SQL:

- **001_init_schema.sql** - cria schema, tipos, tabelas, índices e triggers
- **002_supabase_security.sql** - hardening de acesso (opcional mas recomendado)
- **003_seed_dev.sql** - seed para dev (opcional)

4.1 Executar pelo Supabase SQL Editor

No painel do Supabase, abra **SQL Editor** e rode na ordem:

```
-- 1) schema
-- cole o conteúdo de 001_init_schema.sql e execute

-- 2) segurança (recomendado)
-- cole o conteúdo de 002_supabase_security.sql e execute

-- 3) seed dev (opcional)
-- cole o conteúdo de 003_seed_dev.sql e execute
```

4.2 Executar via psql (Debian/macOS/Windows WSL)

Use a connection string do Supabase (Database settings). Exemplo:

```
export DB_URL="postgresql://postgres:<SENHA>@db.<PROJECT_REF>.supabase.co:5432/postgres?sslmode=verify-full"
psql "$DB_URL" -f 001_init_schema.sql
```

```
psql "$DB_URL" -f 002_supabase_security.sql  
psql "$DB_URL" -f 003_seed_dev.sql
```

5. Segurança recomendada (Supabase)

Mesmo que o backend seja a única camada que usa o banco, o Supabase expõe APIs REST/Realtime por padrão. Para evitar acesso indevido por chaves públicas:

- 1) Revogar permissões de anon e authenticated.
- 2) Ativar RLS em todas as tabelas (sem policies -> acesso negado).
- 3) Operar via `service_role` no backend.

O script **002_supabase_security.sql** faz exatamente isso.

Observação: o `service_role` bypassa RLS. Portanto, o backend mantém acesso total, enquanto as chaves públicas ficam bloqueadas.

6. Boas práticas de migrations no repositório

Recomendação de organização:

```
db/  
  migrations/  
    001_init_schema.sql  
    002_supabase_security.sql  
    003_seed_dev.sql  
  README_DB.md
```

Regras:

- Nunca editar migrations já aplicadas em produção. Crie uma nova (ex.: `004_add_index_x.sql`).
- Toda alteração deve ser revisada por PR, com diff do SQL e checklist de impacto.
- Em produção: aplicar migrations em janela de manutenção quando houver ALTER TABLE pesado.

7. Validações e smoke tests

Após aplicar o schema, rode consultas rápidas para validar:

```
-- ver tabelas  
SELECT tablename FROM pg_tables WHERE schemaname='public' ORDER BY tablename;  
  
-- ver enums criados  
SELECT t.typname FROM pg_type t WHERE t.typtype='e' ORDER BY t.typname;  
  
-- checar unicidade de instance_key  
SELECT instance_key, count(*) FROM instances GROUP BY 1 HAVING count(*) > 1;  
  
-- checar 1 thread por unidade  
SELECT instance_id, unit_id, count(*) FROM inbox_threads GROUP BY 1,2 HAVING count(*) > 1;  
  
-- checar 1 turno ativo por entregador  
SELECT instance_id, user_id, count(*)  
FROM delivery_turns  
WHERE ended_at IS NULL  
GROUP BY 1,2 HAVING count(*) > 1;
```

8. Checklist de '100% pronto'

Item	OK quando...
Schema aplicado	001_init_schema.sql executado sem erros e tabelas/enums existentes
Storage pronto	bucket attachments criado como Private e backend consegue upload

Segurança	002_supabase_security.sql aplicado e anon/authenticated sem acesso às tabelas
Seed dev (opcional)	003_seed_dev.sql executado e usuários dev conseguem logar no backend
Índices	principais índices por instance_id/status/datas criados (ver pg_indexes)
Auditoria	audit_logs bloqueia update/delete (trigger ativo)
Back-end conectado	backend consegue: criar usuário, login, refresh, CRUD de unidade/tickets/entregas, anexos

Fim do documento.